

Milestone 2 Progress Report: NeuroFleetX

Project Title: NeuroFleetX - Smart Fleet Management System

Phase: Backend Development & API Integration

1. Executive Summary

During Milestone 2, the primary focus was transitioning from a static frontend to a dynamic, data-driven application. We successfully established a robust **Spring Boot** backend, implemented core CRUD (Create, Read, Update, Delete) operations, and integrated these services with the **React.js** frontend using **Axios**.

2. Backend Architectural Implementation

We adopted a layered architecture in Spring Boot to ensure scalability and maintainability of the NeuroFleetX system:

- **Layered Structure:** Implemented Controller, Service, Repository, and Entity layers for a clean separation of concerns.
- **Database Integration:** Configured **MongoDB** as the primary database to handle flexible data models for Buses, Drivers, and Routes.
- **RESTful APIs:** Developed endpoints for:
 - **Fleet Management:** Adding, updating, and viewing bus details including capacity and status.
 - **Driver Coordination:** Managing driver profiles and assignments.
 - **Route Mapping:** Storing route names and intermediate stops.

3. Frontend Development & Integration

The frontend was enhanced to consume real-time data from the backend:

- **API Interceptor:** Created a centralized Axios configuration (api.js) to handle base URLs and request/response interceptors for seamless communication.
- **Passenger Dashboard Integration:** * Developed the **PassengerTicketBooking** component to fetch real-time bus availability.
 - Implemented logic to handle data mismatches (e.g., fallback for totalSeats and capacity fields).
 - Added **Safe Rendering** techniques to prevent UI crashes when dealing with optional array data like stops.

4. Testing and Verification

- **API Testing:** All backend endpoints were rigorously tested using **Postman** to verify JSON responses and HTTP status codes.
- **Cross-Origin Resource Sharing (CORS):** Configured `@CrossOrigin` annotations in Spring Boot to allow the React frontend (running on port 5173) to communicate with the backend.
- **Error Handling:** Implemented frontend validation to ensure users select a valid date and trip before booking.

5. Challenges Overcome

- **Dynamic Data Binding:** Resolved issues where frontend variables did not match backend JSON keys by implementing conditional mapping.
- **Undefined Property Errors:** Fixed `TypeError` issues during array joining by using optional chaining and null checks.

6. Next Steps for Milestone 3

- Implementation of **MyTickets.jsx** for personal booking history.
- Integration of **Advanced Table Relationships** (linking Bus and Route collections).
- Live location tracking simulation.